

C++ 프로그래밍

함수 포인터 (Function Pointer)

함수 포인터란?

함수도 메모리 주소를 갖는다. **함수 포인터**는 그 주소를 저장하는 변수다.

선언 · 대입 · 호출

```
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

// 선언: 반환타입 (*이름)(매개변수 타입...)
int (*op)(int, int);

op = add;                // 대입 - & 생략 가능
printf("%d\n", op(3, 4)); // 호출 → 7

op = sub;
printf("%d\n", op(3, 4)); // 호출 → -1

// 선언과 동시에 초기화
int (*fp)(int, int) = add;
```

포인터와의 비교

	데이터 포인터	함수 포인터
선언	<code>int* p</code>	<code>int (*fp)(int)</code>
대입	<code>p = &x</code>	<code>fp = func</code>
역참조	<code>*p</code>	<code>(*fp)(arg)</code> 또는 <code>fp(arg)</code>
용도	데이터 접근	함수 간접 호출

콜백 함수 (Callback Function)

콜백이란 "나중에 불러달라고 넘겨주는 함수"다.

호출자(caller)가 피호출자(callee)에게 함수를 전달하면, 피호출자가 적절한 시점에 그 함수를 호출한다.

활용 예시

상황	콜백 역할
<code>qsort</code>	정렬 기준 (비교 함수)
GUI 이벤트	버튼 클릭 시 실행할 함수
비동기 I/O	작업 완료 후 호출할 함수
알고리즘 라이브러리	각 원소에 적용할 처리 함수

콜백의 핵심: "무엇을 할지"를 함수로 분리해 전달 — 호출 시점과 호출 대상을 분리한다.

함수 포인터를 매개변수로 — 콜백

함수를 인자로 전달해 **호출 시점에 동작을 교체**한다.

아래 예제에서 `apply` 함수는 바뀌지 않는다.

다만, **전달하는 콜백만 교체**해 `apply` 함수의 동작 원리를 바꾸는 것이다.

콜백 함수

```
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

// op 자리에 어떤 함수든 전달 가능
int apply(int a, int b, int (*op)(int, int)) {
    return op(a, b);
}

printf("%d\n", apply(3, 4, add)); // 7
printf("%d\n", apply(3, 4, mul)); // 12
```

콜백의 흐름

```
apply(3, 4, add)
├─ a=3, b=4 고정
├─ op = add ← 콜백으로 전달
└─ add(3, 4) → 7

apply(3, 4, mul)
├─ op = mul ← 다른 콜백 전달
└─ mul(3, 4) → 12
```

qsort — 콜백 기반 정렬

qsort 는 비교 함수를 콜백으로 받아 정렬 기준을 런타임에 결정한다.

함수 시그니처

```
// <stdlib>
void qsort(
    void* base,    // 배열 시작 주소
    size_t count,  // 원소 개수
    size_t size,   // 원소 하나의 크기 (bytes)
    int (*cmp)(const void*, const void*) // 비교 콜백
);
```

비교 콜백 반환값 규칙

반환값	의미	결과
음수	<code>a < b</code>	a를 앞에 배치
0	<code>a == b</code>	순서 유지
양수	<code>a > b</code>	b를 앞에 배치

콜백 등록 예제

```
#include <stdlib>
#include <stdio>

int cmp_asc(const void* a, const void* b) {
    return *(int*)a - *(int*)b;
    // 음수 → a 앞, 양수 → b 앞 (오름차순)
}

int cmp_desc(const void* a, const void* b) {
    return *(int*)b - *(int*)a;
    // 반대 방향 (내림차순)
}

int arr[] = { 5, 2, 8, 1, 9 };

qsort(arr, 5, sizeof(int), cmp_asc);    // 콜백 등록
// arr → 1 2 5 8 9

qsort(arr, 5, sizeof(int), cmp_desc);   // 다른 콜백 등록
// arr → 9 8 5 2 1
```

qsort 내부는 그대로다. 콜백만 바꿔 정렬 기준을 교체한다.

typedef — 가독성 개선 (C 스타일)

typedef 로 함수 포인터 타입에 이름을 붙이면 선언이 간결해진다.

❌ typedef 없이

```
#include <stdio>

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int apply(int a, int b,
          int (*op)(int, int)) {
    return op(a, b);
}

int main() {
    int (*op)(int, int) = add;
    printf("%d\n", op(3, 4));           // 7
    printf("%d\n", apply(3, 4, mul));  // 12

    int (*table[2])(int, int) = { add, mul };
    printf("%d\n", table[0](2, 3));    // 5
    printf("%d\n", table[1](2, 3));    // 6
}
```

✅ typedef 사용

```
#include <stdio>

typedef int (*BinaryOp)(int, int);

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int apply(int a, int b, BinaryOp op) {
    return op(a, b);
}

int main() {
    BinaryOp op = add;
    printf("%d\n", op(3, 4));           // 7
    printf("%d\n", apply(3, 4, mul));  // 12

    BinaryOp table[2] = { add, mul };
    printf("%d\n", table[0](2, 3));    // 5
    printf("%d\n", table[1](2, 3));    // 6
}
```

using — 가독성 개선 (C++11)

C++11의 `using` 은 `typedef` 보다 읽는 방향이 자연스럽다.

별칭 = 타입 형태로 일반 변수 선언과 동일한 방향으로 읽힌다.

✗ using 없이 (typedef)

```
#include <cstdio>

typedef int (*BinaryOp)(int, int);

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int apply(int a, int b, BinaryOp op) {
    return op(a, b);
}

int main() {
    BinaryOp op = add;
    printf("%d\n", op(3, 4));           // 7
    printf("%d\n", apply(3, 4, mul));  // 12

    BinaryOp table[2] = { add, mul };
    printf("%d\n", table[0](2, 3));    // 5
    printf("%d\n", table[1](2, 3));    // 6
}
```

✓ using 사용

```
#include <cstdio>

using BinaryOp = int (*)(int, int);
//      ^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^
//      별칭이름      = 실제 타입 (좌→우로 읽힘)

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int apply(int a, int b, BinaryOp op) {
    return op(a, b);
}

int main() {
    BinaryOp op = add;
    printf("%d\n", op(3, 4));           // 7
    printf("%d\n", apply(3, 4, mul));  // 12

    BinaryOp table[2] = { add, mul };
    printf("%d\n", table[0](2, 3));    // 5
    printf("%d\n", table[1](2, 3));    // 6
}
```

C++ 클래스와 qsort

qsort 비교 콜백에서 `const void*` 를 클래스 포인터로 캐스팅해 멤버에 접근한다.

클래스 & 비교 함수

```
#include <cstdlib>
#include <cstdio>
#include <cstring>

class Student {
public:
    char name[20];
    int score;
    Student(const char* _name, int _score)
        : score(_score) {
        strncpy(name, _name, 20);
    }
};

int cmp_by_score(const void* a, const void* b) {
    const Student* sa = (const Student*)a;
    const Student* sb = (const Student*)b;
    return sb->score - sa->score; // 내림차순
}

int cmp_by_name(const void* a, const void* b) {
    const Student* sa = (const Student*)a;
    const Student* sb = (const Student*)b;
    return strcmp(sa->name, sb->name); // 오름차순
}
```

main — 콜백 교체로 정렬 기준 변경

```
int main() {
    Student arr[] = {
        Student("Alice", 85), Student("Bob", 92), Student("Carol", 78)
    };

    qsort(arr, 3, sizeof(Student), cmp_by_score);
    for (int i = 0; i < 3; i++)
        printf("%s: %d\n", arr[i].name, arr[i].score);
    // Bob: 92
    // Alice: 85
    // Carol: 78

    qsort(arr, 3, sizeof(Student), cmp_by_name);
    for (int i = 0; i < 3; i++)
        printf("%s: %d\n", arr[i].name, arr[i].score);
    // Alice: 85
    // Bob: 92
    // Carol: 78
}
```

`const void*` → `const Student*` 캐스팅으로 클래스 멤버에 접근한다.

std::sort + operator<

operator< 는 멤버 함수 또는 전역 함수 두 가지 방식으로 정의할 수 있다.

멤버 함수로 정의

```
#include <algorithm>
#include <cstdio>
#include <cstring>

class Student {
public:
    char name[20];
    int score;
    Student(const char* _name, int _score)
        : score(_score) { strncpy(name, _name, 20); }

    // 왼쪽 피연산자가 *this
    bool operator<(const Student& o) const {
        return score < o.score;
    }
};

int main() {
    Student arr[] = {
        Student("Alice", 85), Student("Bob", 92), Student("Carol", 78)
    };
    std::sort(arr, arr + 3);
    for (int i = 0; i < 3; i++)
        printf("%s: %d\n", arr[i].name, arr[i].score);
    // Carol: 78 Alice: 85 Bob: 92
}
```

전역 함수로 정의

```
#include <algorithm>
#include <cstdio>
#include <cstring>

class Student {
public:
    char name[20];
    int score;
    Student(const char* _name, int _score)
        : score(_score) { strncpy(name, _name, 20); }
};

// 두 피연산자를 모두 매개변수로 받음
bool operator<(const Student& a, const Student& b) {
    return a.score < b.score;
}

int main() {
    Student arr[] = {
        Student("Alice", 85), Student("Bob", 92), Student("Carol", 78)
    };
    std::sort(arr, arr + 3);
    for (int i = 0; i < 3; i++)
        printf("%s: %d\n", arr[i].name, arr[i].score);
    // Carol: 78 Alice: 85 Bob: 92
}
```

람다로 정렬 기준 교체

람다를 사용하면 클래스 수정 없이 호출 시점에 정렬 기준을 자유롭게 바꿀 수 있다.

score 기준 정렬

```
#include <algorithm>
#include <cstdio>
#include <cstring>

class Student {
public:
    char name[20];
    int score;
    Student(const char* _name, int _score)
        : score(_score) { strncpy(name, _name, 20); }
};

int main() {
    Student arr[] = {
        Student("Alice", 85), Student("Bob", 92), Student("Carol", 78)
    };

    std::sort(arr, arr + 3, // 오름차순
        [](const Student& a, const Student& b) {
            return a.score < b.score;
        });
    // Carol: 78 Alice: 85 Bob: 92

    std::sort(arr, arr + 3, // 내림차순
        [](const Student& a, const Student& b) {
            return a.score > b.score;
        });
}
```

name 기준 정렬

```
#include <algorithm>
#include <cstdio>
#include <cstring>

class Student {
public:
    char name[20];
    int score;
    Student(const char* _name, int _score)
        : score(_score) { strncpy(name, _name, 20); }
};

int main() {
    Student arr[] = {
        Student("Alice", 85), Student("Bob", 92), Student("Carol", 78)
    };

    std::sort(arr, arr + 3, // 오름차순
        [](const Student& a, const Student& b) {
            return strcmp(a.name, b.name) < 0;
        });
    // Alice: 85 Bob: 92 Carol: 78

    std::sort(arr, arr + 3, // 내림차순
        [](const Student& a, const Student& b) {
            return strcmp(a.name, b.name) > 0;
        });
}
```

함수 포인터와 훅 (Hook)

훅이란 기존 코드의 특정 시점에 **사용자 함수를 끼워 넣는** 패턴이다. 함수 포인터가 훅 포인트(hook point) 역할을 한다.

커스텀 훅 패턴

```
#include <stdio>

using HookFn = void (*)(const char*);

HookFn on_before = nullptr; // 훅 미설치 시 아무것도 안 함
HookFn on_after  = nullptr;

void process(const char* data) {
    if (on_before) on_before(data); // 전처리 훅 호출
    printf("처리 중: %s\n", data);
    if (on_after)  on_after(data);  // 후처리 훅 호출
}

void log_before(const char* d) { printf("[시작] %s\n", d); }
void log_after (const char* d) { printf("[완료] %s\n", d); }
```

```
int main() {
    process("A");           // 훅 없이 실행
    // 처리 중: A

    on_before = log_before; // 훅 설치
    on_after  = log_after;
    process("B");           // 훅과 함께 실행
    // [시작] B
    // 처리 중: B
    // [완료] B
}
```

함수 포인터와 훅 (Hook)

표준 라이브러리에도 훅이 있음.

훅의 핵심: 호출 시점은 라이브러리/프레임워크가 결정하고, 무엇을 할지는 사용자가 함수 포인터로 전달한다.

std::atexit — 프로그램 종료 훅

```
#include <cstdlib>
#include <cstdio>

void cleanup1() { printf("cleanup1 실행\n"); }
void cleanup2() { printf("cleanup2 실행\n"); }

int main() {
    std::atexit(cleanup2); // 종료 시 호출될 함수 등록
    std::atexit(cleanup1);

    printf("main 종료\n");
}
// 출력 (LIFO 순서):
// main 종료
// cleanup1 실행
// cleanup2 실행
```

std::set_new_handler — 메모리 할당 실패 훅

```
#include <new>
#include <cstdlib>
#include <cstdio>

void on_alloc_fail() {
    printf("메모리 부족!\n");
    std::abort();
}

int main() {
    // operator new 실패 시 호출될 훅 설치
    std::set_new_handler(on_alloc_fail);
}
```

함수 포인터와 가상 함수

virtual 함수는 내부적으로 **함수 포인터 배열(vtable)**로 구현된다. 수동으로 함수 포인터를 관리하는 것을 컴파일러가 자동화한 것이다.

함수 포인터로 직접 구현 (수동)

```
#include <stdio>

struct Animal {
    const char* name;
    void (*speak)(const char*); // 함수 포인터 멤버
};

void dog_speak(const char* n) { printf("%s: 왈왈!\n", n); }
void cat_speak(const char* n) { printf("%s: 야옹!\n", n); }

int main() {
    Animal dog = { "멍멍이", dog_speak };
    Animal cat = { "냥냥이", cat_speak };

    Animal* zoo[] = { &dog, &cat };
    for (auto* a : zoo)
        a->speak(a->name); // 함수 포인터로 직접 디스패치
    // 멍멍이: 왈왈!
    // 냥냥이: 야옹!
}
```

virtual 함수 (컴파일러 자동 관리)

```
#include <stdio>

class Animal {
public:
    const char* name;
    Animal(const char* _name) : name(_name) {}
    virtual void speak() const = 0;
};

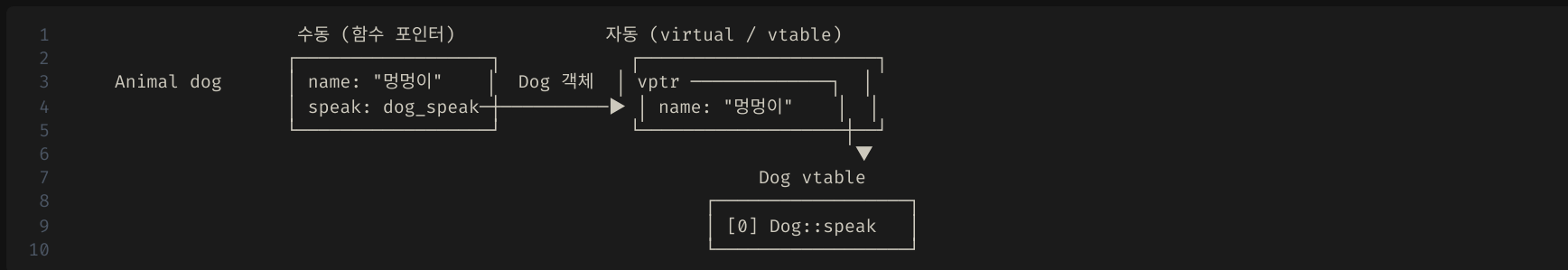
class Dog : public Animal {
public:
    Dog(const char* _name) : Animal(_name) {}
    void speak() const override { printf("%s: 왈왈!\n", name); }
};

class Cat : public Animal {
public:
    Cat(const char* _name) : Animal(_name) {}
    void speak() const override { printf("%s: 야옹!\n", name); }
};

int main() {
    Dog dog("멍멍이");
    Cat cat("냥냥이");
```

함수 포인터 vs vtable

`virtual` 함수를 가진 클래스는 컴파일러가 **`**vtable(가상 함수 테이블)**`**을 자동 생성한다. `vtable`은 본질적으로 **함수 포인터 배열**이다.



	함수 포인터 (수동)	<code>virtual</code> 함수 (자동)
테이블 관리	개발자가 직접	컴파일러 자동 생성
타입 안전성	없음 (<code>void*</code> 등 위험)	있음
상속 연동	없음	파생 클래스 자동 반영
오버헤드	동일 (포인터 1회 역참조)	동일 (포인터 1회 역참조)

`virtual` 함수는 함수 포인터 디스패치를 **타입 시스템과 상속에 통합**한 것이다.